**The University Of Azad Jammu &**

**Kashmir ,Muzaffarabad**

**Class Activity**

**FROM REAL WORLD CONCEPT TO SOFTWARE OBJECT**

**Submission Details**

| | |
|---|---|
| **Student Name** | Misbah Khurshid |
| **Roll no** | 2025-SE-10 |
| **Course Code** | SEC-532 |
| **Course Name** | Software Design &Architecture |
| **Lab Title** | From Real World Concept to Software Object (Food Delivery System) |
| **University** | The University of Azad Jammu and Kashmir |



# Food Delivery System: Conceptual Classes and Abstraction

## 1. Introduction and Selected Domain:

For this assignment I selected the Food Delivery System as my real world domain. A food delivery service has to look after many things at the same time, such as customers, restaurants, menus, orders, payments, delivery riders, delivery addresses, promotions and customer reviews. When these are handled without a proper structure, orders get delayed, food arrives cold or incorrect, and information gets lost. The purpose of this system is to organize these activities in one place so that the journey from placing an order to receiving the food becomes easier to manage.

## 2. Candidate Conceptual Classes:

I identified the following 10 conceptual classes. They are conceptual classes because each one stands for a meaningful entity or idea that exists in the food delivery domain, and not for a screen, a table or a piece of code.

1. **Customer:** A person who orders food through the service. It stores contact information and helps track order history and preferences.
2. **Restaurant:** A business that prepares and sells food. It stores the restaurant name, location, opening hours and the menu it offers.
3. **MenuItem:** A single dish or drink offered by a restaurant. It records the item name, description, price and availability.
4. **Order:** A request placed by a customer for one or more menu items. It tracks the items ordered, total amount and current status.
5. **OrderItem:** One line within an order, linking a menu item with the quantity requested. It helps calculate the cost of each line.
6. **DeliveryRider:** A person who collects the order from the restaurant and delivers it to the customer. It stores rider details, availability and current delivery status.
7. **DeliveryAddress:** The location where an order must be delivered. It stores the street, area, city and contact instructions.
8. **Payment:** Money paid by the customer for an order. It records the amount, date, payment method and payment status.
9. **Promotion:** A discount or offer, such as a voucher code. It stores the discount value, validity period and conditions of use.
10. **Review:** Feedback given by a customer about a restaurant or delivery. It stores the rating, comment and date.

# 3. Attributes and Methods of Three Important Classes:

Attributes are the pieces of information a class stores, while methods are the actions or behavior the class can perform. I chose Customer, Order and Restaurant for detailed modeling.

## 3.1 Customer Class:

**Attributes:**

- **customerId:** A unique identifier used to distinguish one customer from another.
- **fullName:** Stores the customer's complete name.
- **email:** Stores the customer's email address for notifications and receipts.
- **phoneNumber:** Stores the customer's contact number for delivery coordination.
- **defaultAddress:** Identifies the address most commonly used for deliveries.

**Methods:**

- **updateProfile():** Updates the customer's personal details.
- **browseRestaurants():** Displays the restaurants available to the customer.

- **placeOrder():** Allows the customer to create a new food order.
- **viewOrderHistory():** Displays the customer's previous orders.

### 3.2 Order Class:

**Attributes:**

- **orderId:** A unique identifier for each order.
- **customerId:** Identifies the customer who placed the order.
- **restaurantId:** Identifies the restaurant preparing the order.
- **orderDate:** Records the date and time when the order was placed.
- **totalAmount:** Stores the final amount payable, including delivery charges and discounts.
- **status:** Shows the current progress, such as placed, preparing, out for delivery or delivered.

**Methods:**

- **createOrder():** Creates a new order record.
- **updateStatus():** Changes the order's current status.
- **assignRider():** Assigns an available delivery rider to the order.
- **calculateTotal():** Calculates the total using item prices, quantities, delivery fee and any promotion.

### 3.3 Restaurant Class:

**Attributes:**

- **restaurantId:** A unique identifier for each restaurant.
- **restaurantName:** Stores the name of the restaurant.
- **location:** Records where the restaurant is situated.
- **cuisineType:** Identifies the type of food served, such as fast food, desi or Chinese.
- **openingHours:** Stores the times during which the restaurant accepts orders.
- **rating:** Shows the average rating based on customer reviews.

**Methods:**

- **addMenuItem():** Adds a new dish to the restaurant's menu.
- **updateMenuItem():** Updates the details or price of an existing dish.
- **acceptOrder():** Confirms that the restaurant will prepare an incoming order.
- **generateSalesSummary():** Produces a summary of the restaurant's orders for review.

## 4. Main Abstraction Decisions:

Customer, Order and Restaurant were selected for detailed modeling because they are central to the daily operations of a food delivery service. The other classes act as supporting concepts that help manage

menus, deliveries, payments, offers and feedback. Real-time GPS tracking, route optimization, rider salary calculation, inventory management and detailed accounting were deliberately excluded because they are beyond the scope of this small model. Screens, database tables, login controllers and programming language details were also left out, since conceptual modeling focuses on the real world business domain and not on technical implementation.

## 5. Invented Software Object:

### Order Validation Checker:

An Order Validation Checker is an invented software object. It does not represent a physical entity in the food delivery business, but it is still useful in the design.

- **Purpose:** Checks an order before it is sent to the restaurant.
- **Validation:** Checks whether the cart contains at least one item, the restaurant is open, every item is available, a valid delivery address is provided and the payment method is selected.
- **Result:** Returns whether the order can proceed or identifies the missing or invalid information.
- **Benefit:** Keeps validation rules consistent and reduces incomplete, failed or incorrect orders.
- Even though it has no direct counterpart in the real world, this object shows that software design sometimes needs objects created only to coordinate processes or enforce business rules.

## 6. Conclusion:

The Food Delivery System is a practical example of conceptual modeling. By identifying classes, defining their attributes and methods, and making clear abstraction decisions, we can organize a software system around real business needs before thinking about implementatio